

Systemes Intelligents Avancés

Manon Ansart

Objectif : Concevoir et entraîner un réseau de neurones convolutif (CNN) pour la classification d'images sur le jeu de données MNIST.

Ce réseau va être créé en PyTorch. Vous pouvez reprendre les TP de l'année dernière ou les tutoriels Pytorch pour vous aider.

1 Réseau de base

1. Charger et prétraiter le jeu de données MNIST à l'aide de Pytorch. Regardez les caractéristiques du jeu de données. À quoi correspond-t-il ? Combien y a-t-il d'images ? Quelle est leur taille ?
2. Construire un CNN simple avec l'architecture suivante :
 - Une couche de convolution avec 10 filtres de taille 5x5 avec padding same.
 - Une fonction d'activation ReLU.
 - Une couche de max pooling avec un noyau de taille 2x2 et un striding de 2.
 - Une couche entièrement connectée avec 300 neurones (taille d'entrée à déterminer).
 - Une fonction d'activation ReLU.
 - Une couche entièrement connectée avec un nombre de neurones adapté à la sortie.
 - Une fonction softmax pour la classification.

Quelle est la taille de la feature map après la couche de convolution ? Après la couche de pooling ? Combien d'entrées la couche fully connected suivante a-t-elle donc ?

3. Utiliser la fonction `summary` du package `torchsummary` pour afficher le nombre de paramètre par couche et le nombre de paramètres total.
4. Entraîner le modèle en utilisant l'optimiseur Adam et la fonction de coût cross-entropy sur les données d'apprentissage. Vous utiliserez 2 epochs et un batch_size de 64 pour commencer.
5. Évaluer le modèle sur l'ensemble de test et afficher l'accuracy.
6. Tracer la courbe de loss en fonction des epoch d'entraînement.

2 Amélioration du modèle

1. Expérimenter avec différentes architectures de réseau :
 - Augmenter le nombre de couches de convolution.
 - Ajouter du drop-out
 - Varier le nombre de filtres et leur taille.
 - Ajouter des couches de batch normalization.
2. Faites varier les hyperparamètres du modèle :

- Taux d'apprentissage.
 - Taille des batchs.
 - Nombre d'époques d'entraînement.
 - Taux de dropout.
3. Comparer les performances des différents modèles et identifier les architectures et les hyperparamètres qui donnent les meilleurs résultats.